

BudgetTrackerAI: Eine minimalistische Budget-Tracker Mobile-App mit KI-Finanzassistent basierend auf React Native für plattformübergreifende Nutzung

Bachelor-IT Projekt

Amanullah Naderi

Fachhochschule Erfurt

Betreuerin: Frau Prof. Birgit Hebestreit

24. Juni 2025

Contents

1	Einleitung	3
2	Systemarchitektur	3
2.1	Überblick	3
2.2	Frontend-Architektur	3
2.3	Backend-Architektur	4
2.4	Datenfluss	4
3	Konfiguration	5
3.1	Frontend-Einrichtung	5
3.2	Backend-Einrichtung	6
4	API-Endpunkte	7
5	Wichtige Komponenten	7
5.1	Frontend-Komponenten	7
5.1.1	App.tsx	7
5.1.2	FinancialPieChart.tsx	7
5.1.3	AuthLayout.tsx	8
5.1.4	Button.tsx	8
5.1.5	Input.tsx	8
5.2	Hooks	8
5.2.1	useAuth.ts	8
5.2.2	useBudgets.ts	8
5.2.3	useInsights.ts	9
5.2.4	useTransactions.ts	9
5.2.5	useUserRole.ts	9
5.3	Dienste	9
5.3.1	api.ts	9
5.3.2	auth.ts	9
5.3.3	database.ts	9
5.3.4	firebase.ts	10
5.3.5	user.ts	10
5.3.6	userProfile.ts	10
5.4	Bildschirme	10
5.4.1	LoginScreen.tsx	10
5.4.2	RegisterScreen.tsx	10

5.4.3	ResetPasswordScreen.tsx	11
5.4.4	DashboardScreen.tsx	11
5.4.5	TransactionsScreen.tsx	11
5.4.6	BudgetsScreen.tsx	11
5.4.7	InsightsScreen.tsx	11
5.4.8	SettingsScreen.tsx	12
5.4.9	AdminPanelScreen.tsx	12
6	Datenmodelle	12
6.1	Transaktion	12
6.2	Budget	12
6.3	UserData	13
6.4	ConversationMessage	13
7	Einschränkungen und zukünftige Verbesserungen	13
7.1	Einschränkungen	13
7.2	Zukünftige Verbesserungen	14
8	Schlussfolgerung	14

1 Einleitung

BudgetTrackerAI ist eine minimalistische, plattformübergreifende Mobile-Anwendung, die entwickelt wurde, um Nutzern eine effiziente Verwaltung ihrer persönlichen Finanzen zu ermöglichen. Die App wurde mit React Native und Expo erstellt und integriert ein Firebase-Backend für Authentifizierung und Datenspeicherung sowie eine benutzerdefinierte Node.js/Express-API für KI-gestützte Finanzanalysen. Sie unterstützt iOS, Android und Webplattformen und bietet Funktionen wie Transaktionsverfolgung, Budgetplanung, KI-basierte Finanzberatung und Benutzerverwaltung. Diese Dokumentation bietet eine umfassende Übersicht über die Architektur, Konfiguration, Endpunkte und Hauptkomponenten der Frontend- und Backendsysteme und dient als technischer Leitfaden für die von Amanullah Naderi an der Fachhochschule Erfurt eingereichte Bachelorarbeit.

2 Systemarchitektur

2.1 Überblick

BudgetTrackerAI folgt einer Client-Server-Architektur mit einem React Native-Frontend und einem Node.js/Express-Backend, das Firebase für Authentifizierung und Firestore für Datenpersistenz nutzt. Das Frontend bietet eine reaktionsschnelle Benutzeroberfläche, während das Backend API-Anfragen verarbeitet und KI-gestützte Analysen generiert. Echtzeit-Datensynchronisation wird durch Firestore-Abonnements erreicht.

2.2 Frontend-Architektur

Das Frontend basiert auf React Native mit Expo und verwendet TypeScript für Typsicherheit. Wichtige Aspekte sind:

- **Framework:** React Native mit Expo für plattformübergreifende Entwicklung.
- **Styling:** Tailwind CSS über NativeWind für responsives Design.
- **Navigation:** React Navigation mit Stack- und Bottom-Tab-Navigatoren.
- **Zustandsverwaltung:** Benutzerdefinierte Hooks (useAuth, useTransactions, useBudgets, useInsights) für Zustandsmanagement.
- **Datendarstellung:** React Native Chart Kit (Kreisdiagramme) und Recharts (Liniendiagramme).

- **Abhängigkeiten:** Firebase, expo-haptics, expo-speech, expo-document-picker, expo-linear-gradient.

2.3 Backend-Architektur

Das Backend wird als Node.js/Express-Server angenommen mit folgenden Komponenten:

- **Framework:** Node.js mit Express für die Verarbeitung von API-Anfragen.
- **Datenbank:** Firebase Firestore für die Speicherung von Transaktionen, Budgets und Benutzerdaten.
- **Authentifizierung:** Firebase Admin SDK zur Verifizierung von Benutzertoken.
- **KI-Integration:** Benutzerdefinierter API-Endpunkt zur Verarbeitung von Finanzdaten und Generierung von Analysen (vermutlich mit einem Machine-Learning-Modell).
- **Bereitstellung:** Lokal oder über einen Cloud-Dienst gehostet, zugänglich über eine konfigurierte URL (z.B. `http://192.168.0.122:3000`).

2.4 Datenfluss

Der Datenfluss umfasst:

1. Benutzerauthentifizierung über Firebase Authentication, mit Rollen in Firestore gespeichert.
2. Speicherung von Transaktions- und Budgetdaten in Firestore, verknüpft mit Benutzer-IDs.
3. Echtzeit-Updates durch Firestore onSnapshot-Abonnements.
4. Generierung von Finanzanalysen durch Übermittlung von Transaktionsdaten an die Backend-API.
5. Visualisierung der Daten mit Diagrammen und Fortschrittsbalken im Frontend.

3 Konfiguration

3.1 Frontend-Einrichtung

1. **Voraussetzungen:** Node.js (v18+), Yarn oder npm, Expo CLI (npm install -g expo-cli), Firebase-Projekt.

2. **Repository klonen:**

```
git clone <repository-url>  
cd BudgetTrackerAI
```

3. **Abhängigkeiten installieren:**

```
yarn install
```

4. **Umgebungsvariablen konfigurieren:** Eine .env-Datei erstellen:

```
EXPO_PUBLIC_API_URL=http://192.168.0.122:3000  
EXPO_PUBLIC_API_URL_WEB=http://localhost:3000  
EXPO_PUBLIC_FIREBASE_API_KEY=AIzaSyATH7IANLMmUt6gFU0a-  
  WaIIJUc8ZVRco8  
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=budgettrackerai-7d3d3.  
  firebaseapp.com  
EXPO_PUBLIC_FIREBASE_PROJECT_ID=budgettrackerai-7d3d3  
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=budgettrackerai-7d3d3.  
  firebasestorage.app  
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=631236623635  
EXPO_PUBLIC_FIREBASE_APP_ID=1:631236623635:web:  
  e0f0442cb46f76d6b1eee6
```

5. **Anwendung starten:**

```
yarn start
```

Verwenden Sie Expo Go für Mobilgeräte oder wählen Sie die Weboption in der Expo CLI.

6. **Tunneling (optional):** Für physische Geräte:

```
npx expo start --tunnel
```

Aktualisieren Sie `EXPO_PUBLIC_API_URL` mit der *ngrok* – URL.

3.2 Backend-Einrichtung

1. **Voraussetzungen:** Node.js (v18+), Firebase-Projekt mit Authentifizierung und Firestore.

2. **Firebase einrichten:**

- Erstellen Sie ein Projekt unter <https://console.firebase.google.com>.
- Aktivieren Sie E-Mail/Passwort-Authentifizierung und Firestore.
- Fügen Sie die Firebase-Konfiguration zur `.env`-Datei hinzu.

3. **API bereitstellen:**

- Stellen Sie den Node.js/Express-Server lokal oder über einen Cloud-Dienst bereit.
- Stellen Sie sicher, dass der Server unter `EXPUBLIC_API_URL` erreichbar ist. Konfigurieren Sie den Zugriff.

4. **Firestore-Sicherheitsregeln:**

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth
        .uid == userId;
    }
    match /transactions/{transactionId} {
      allow read, write: if request.auth != null && request.
        resource.data.userId == request.auth.uid;
    }
    match /budgets/{budgetId} {
      allow read, write: if request.auth != null && request.
        resource.data.userId == request.auth.uid;
    }
  }
}
```

4 API-Endpunkte

Das Backend bietet RESTful API-Endpunkte, die über Firebase-ID-Token authentifiziert werden. Die folgende Tabelle fasst die wichtigsten Endpunkte zusammen:

Table 1: Backend-API-Endpunkte

Methode	Endpunkt	Anfragekörper	Antwort
POST	/api/insights	{query: string, userId: string, transactions?: Transaction[]}	{result: string} {error: string}
POST	/api/transactions	Transaktionsdaten (ohne id)	{id: string}
PUT	/api/transactions/:id	Vollständige Transaktionsdaten	Erfolg oder Fehler
DELETE	/api/transactions/:id	Keine	Erfolg oder Fehler
POST	/api/budgets	Budgetdaten (ohne id)	{id: string}
PUT	/api/budgets/:id	Vollständige Budgetdaten	Erfolg oder Fehler
DELETE	/api/budgets/:id	Keine	Erfolg oder Fehler

Alle Endpunkte erfordern ein Firebase-ID-Token im Authorization-Header, das vom Firebase Admin SDK verifiziert wird.

5 Wichtige Komponenten

5.1 Frontend-Komponenten

5.1.1 App.tsx

- **Zweck:** Einstiegspunkt der Anwendung.
- **Funktionen:** Initialisiert SafeAreaProvider, richtet Navigation ein, konfiguriert Statusleiste.

5.1.2 FinancialPieChart.tsx

- **Zweck:** Visualisiert Finanzdaten.
- **Funktionen:** Zeigt Einnahmen, Ausgaben und Saldo mit react-native-chart-kit an. Unterstützt Filterung und Budgetstatusanzeige.
- **Props:** chartData, width, chartConfig, categoryData, showOverallStatus.

5.1.3 *AuthLayout.tsx*

- **Zweck:** Konsistentes Layout für Authentifizierungsseiten.
- **Funktionen:** Zeigt Logo, Titel und handhabt Tastaturvermeidung.
- **Props:** title, children.

5.1.4 *Button.tsx*

- **Zweck:** Wiederverwendbarer Button.
- **Funktionen:** Unterstützt Varianten (primary, secondary, outline), Größen und Ladezustand.
- **Props:** title, onPress, variant, size, loading, disabled, style, textStyle.

5.1.5 *Input.tsx*

- **Zweck:** Wiederverwendbares Texteingabefeld.
- **Funktionen:** Unterstützt Beschriftungen, Platzhalter, Fehlermeldungen und Passwortsichtbarkeit.
- **Props:** label, value, onChangeText, placeholder, secureTextEntry, keyboardType, autoCapitalize, error, toggleSecure, style, textStyle, autoFocus.

5.2 Hooks

5.2.1 *useAuth.ts*

- **Zweck:** Verwaltet Authentifizierungszustand.
- **Funktionen:** Abonniert Firebase-Authentifizierungsänderungen.
- **Rückgabe:** user: User | null, authLoading: boolean.

5.2.2 *useBudgets.ts*

- **Zweck:** Verwaltet Budgetdaten.
- **Funktionen:** Echtzeit-Firestore-Updates, CRUD-Operationen.
- **Rückgabe:** budgets: Budget[], loading: boolean, addBudget, updateBudget, deleteBudget.

5.2.3 *useInsights.ts*

- **Zweck:** Ruft KI-Analysen ab.
- **Funktionen:** Sendet Transaktionsdaten an API, zwischenspeichert Analysen.
- **Rückgabe:** `insights: string[], fetchInsights, clearInsights, loading, error.`

5.2.4 *useTransactions.ts*

- **Zweck:** Verwaltet Transaktionsdaten.
- **Funktionen:** Echtzeit-Updates, Finanzzusammenfassung, CRUD-Operationen.
- **Rückgabe:** `transactions: Transaction[], summary: {income, expenses, balance}, loading, refreshing, onRefresh, addTransaction, updateTransaction, deleteTransaction.`

5.2.5 *useUserRole.ts*

- **Zweck:** Verwaltet Benutzerrollen.
- **Funktionen:** Ruft Rolle aus Firestore ab, handhabt Kontolöschung.
- **Rückgabe:** `role: 'ADMIN' | 'USER' | null, loading, error.`

5.3 Dienste

5.3.1 *api.ts*

- **Zweck:** Verarbeitet Backend-API-Anfragen.
- **Funktionen:** `getApiUrl, getInsights.`

5.3.2 *auth.ts*

- **Zweck:** Verwaltet Firebase-Authentifizierung.
- **Funktionen:** `signUp, signIn, signOut, resetPassword, verifyPasswordResetCode, confirmPasswordReset, subscribeToAuthChanges, getCurrentUser.`

5.3.3 *database.ts*

- **Zweck:** Verwaltet Firestore-Operationen.

- **Funktionen:** addTransaction, updateTransaction, deleteTransaction, addBudget, updateBudget, deleteBudget, subscribeToUserTransactions, subscribeToUserBudgets.

5.3.4 *firebase.ts*

- **Zweck:** Initialisiert Firebase.
- **Funktionen:** Konfiguriert App mit Umgebungsvariablen, exportiert auth, firestore, db.

5.3.5 *user.ts*

- **Zweck:** Verwaltet Benutzerdaten.
- **Funktionen:** getAllUsers, createUser, setUserRole, updateUserRole, deleteUser, getUserRole, checkUserExists, updateUserName.

5.3.6 *userProfile.ts*

- **Zweck:** Verwaltet Benutzerprofile.
- **Funktionen:** getUserProfile, updateUserProfile.

5.4 Bildschirme

5.4.1 *LoginScreen.tsx*

- **Zweck:** Handhabt Benutzeranmeldung und Passwortzurücksetzung.
- **Funktionen:** E-Mail/Passwort-Eingabe, Passwortzurücksetzungsmodal, Toast-Benachrichtigungen.
- **Navigation:** Verlinkt zu Register, ResetPassword.

5.4.2 *RegisterScreen.tsx*

- **Zweck:** Handhabt Benutzerregistrierung.
- **Funktionen:** E-Mail/Passwort/Bestätigungspasswort-Eingabe, setzt USER-Rolle, Toast-Benachrichtigungen.
- **Navigation:** Verlinkt zu Login.

5.4.3 *ResetPasswordScreen.tsx*

- **Zweck:** Verwaltet Passwortzurücksetzung.
- **Funktionen:** Verifiziert Zurücksetzungscode, ermöglicht neue Passworteingabe.
- **Navigation:** Verlinkt zu Login.

5.4.4 *DashboardScreen.tsx*

- **Zweck:** Zeigt Finanzübersicht.
- **Funktionen:** Kreisdiagramm, Transaktionsfilterung, Sparquote, KI-Tipps, benutzerdefinierter Kalender, Benachrichtigungen.
- **Komponenten:** FinancialPieChart, Button, CustomCalendar.

5.4.5 *TransactionsScreen.tsx*

- **Zweck:** Verwaltet Transaktionen.
- **Funktionen:** Transaktionsliste, Hinzufügen/Bearbeiten/Löschen, Filterung, Ausgaben-Liniendiagramm, Finanzzusammenfassung.
- **Komponenten:** Button, Input, MaterialIcons, LinearGradient, LineChart.
- **Navigation:** Bottom-Tab-Navigator.

5.4.6 *BudgetsScreen.tsx*

- **Zweck:** Verwaltet Budgets.
- **Funktionen:** Budgetliste, Hinzufügen/Bearbeiten/Löschen, Fortschrittsbalken, Filterung, Gesamtstatus.
- **Komponenten:** Button, Input, MaterialIcons, LinearGradient.
- **Navigation:** Bottom-Tab-Navigator.

5.4.7 *InsightsScreen.tsx*

- **Zweck:** Bietet KI-gestützte Analysen.
- **Funktionen:** Chat-Oberfläche, Dateianhänge, Nachrichten kopieren/bearbeiten/vorlesen, Tipp-Animation.
- **Komponenten:** MaterialIcons, Feather, LinearGradient, Animated.Image.

- **Navigation:** Bottom-Tab-Navigator.

5.4.8 *SettingsScreen.tsx*

- **Zweck:** Verwaltet Benutzerprofil und Einstellungen.
- **Funktionen:** Profilbearbeitung, Budgetbenachrichtigungen, Hilfe/Support-Modale, Admin-Panel-Zugriff, Abmeldung.
- **Komponenten:** Button, Input, Ionicons, MaterialIcons, CustomCalendar, Toast.
- **Navigation:** Bottom-Tab-Navigator, verlinkt zu AdminPanel.

5.4.9 *AdminPanelScreen.tsx*

- **Zweck:** Verwaltet Benutzerkonten.
- **Funktionen:** Benutzerliste, Suche/Filter/Sortierung, Rollenwechsel, Benutzerlöschung, Echtzeit-Updates.
- **Komponenten:** Button, Ionicons, MaterialIcons, Toast, ScrollableChip.
- **Navigation:** Erreichbar von SettingsScreen.

6 Datenmodelle

6.1 Transaktion

```
type Transaction = {
  id: string;
  userId: string;
  amount: number;
  category: TransactionCategory;
  type: 'income' | 'expense';
  description?: string;
  date: string;
};
```

6.2 Budget

```
type Budget = {
  id: string;
  userId: string;
```

```
category: BudgetCategory;
limit: number;
period: 'weekly' | 'monthly' | 'yearly';
startDate: string | Timestamp;
};
```

6.3 UserData

```
interface UserData {
  uid: string;
  email: string;
  role: 'ADMIN' | 'USER';
  name?: string;
  createdAt?: Timestamp;
}
```

6.4 ConversationMessage

```
type ConversationMessage = {
  id: string;
  role: 'user' | 'ai';
  text: string;
  isEditing?: boolean;
  attachment?: DocumentPickerAsset;
};
```

7 Einschränkungen und zukünftige Verbesserungen

7.1 Einschränkungen

- Kein Offline-Modus; erfordert Internetverbindung.
- Begrenzte Fehlerbehandlung bei Netzwerkausfällen.
- Latenz der Insights-API abhängig von Backend-Verarbeitung.
- Keine Unit-Tests für Komponenten oder Bildschirme.
- Dateianhänge auf 500 Zeichen beschränkt.

7.2 Zukünftige Verbesserungen

- Offline-Unterstützung mit AsyncStorage implementieren.
- Push-Benachrichtigungen für Budgetwarnungen hinzufügen.
- Analysen mit kategorien-spezifischen Trends erweitern.
- Unit-Tests für Hooks, Komponenten und Bildschirme entwickeln.
- Leistung für große Datenmengen optimieren.
- Barrierefreiheit mit Screenreader-Unterstützung und ARIA-Labels verbessern.

8 Schlussfolgerung

BudgetTrackerAI bietet eine minimalistische, aber leistungsstarke Lösung für die persönliche Finanzverwaltung, die ein benutzerfreundliches React Native-Frontend mit einem robusten Firebase- und Node.js-Backend kombiniert. Die plattformübergreifende Unterstützung, Echtzeit-Datensynchronisation und KI-gestützte Analysen machen es zu einem wertvollen Werkzeug für Nutzer, die finanzielle Klarheit suchen. Diese Dokumentation, erstellt für die Bachelorarbeit von Amanullah Naderi unter der Betreuung von Prof. Birgit Hebestreit, bietet eine detaillierte technische Übersicht zur Unterstützung weiterer Entwicklungen und akademischer Bewertung.